

컴퓨터비전개론_과제#2

1. Sobel 에지 검출

본인이 촬영한 영상(크기를 800x800 이하로 조절)에 Sobel 에지 검출을 수행하고 소스 코드와 결과 영상을 제시. 수평과 수직방향 에지 검출 결과와 gradient magnitude로 결합한 결과 영상을 제시

```
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt

img = cv.imread('bed.png')
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

# Sobel 에지 검출
x_edges = cv.Sobel(gray, cv.CV_32F, 1, 0, ksize=3)
y_edges = cv.Sobel(gray, cv.CV_32F, 0, 1, ksize=3)

# 시각화를 위해 8비트로 변환
x_edges_8U = cv.convertScaleAbs(x_edges)
y_edges_8U = cv.convertScaleAbs(y_edges)

# Gradient Magnitude 계산
magnitude = np.sqrt(np.square(x_edges) + np.square(y_edges))

# 시각화
plt.figure(figsize=(9, 6))
plt.subplot(2, 3, 1)
plt.title('Original')
plt.imshow(img)
plt.axis('off')

plt.subplot(2, 3, 4)
plt.title('Sobel X (Horizontal)')
plt.imshow(x_edges_8U, cmap='gray')
plt.axis('off')

plt.subplot(2, 3, 5)
plt.title('Sobel Y (Vertical)')
plt.imshow(y_edges_8U, cmap='gray')
plt.axis('off')

plt.subplot(2, 3, 6)
plt.title('Gradient Magnitude')
plt.imshow(magnitude, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Original



Sobel X (Horizontal)



Sobel Y (Vertical)



Gradient Magnitude



이 코드는 입력 영상 'bed.png'에 대해 Sobel 필터를 이용한 에지 검출을 수행하고, 그 결과를 시각화하는 과정으로 구성되어 있습니다. 먼저 컬러 영상을 그레이스케일로 변환한 뒤, 수평 방향(G_x)과 수직 방향(G_y)의 에지를 각각 Sobel 연산으로 구했습니다. 이렇게 계산된 두 방향의 에지 결과를 시각적으로 보기 위해 절댓값을 취하고 8비트 영상으로 변환하였으며, 이후 G_x 와 G_y 를 이용해 전체 에지 강도를 나타내는 gradient magnitude를 계산했습니다. 마지막으로 원본 영상, 수평 및 수직 에지 영상, 그리고 결합된 에지 강도 영상을 하나의 화면에 출력하여 각각의 단계별 결과를 비교할 수 있도록 구성했습니다.

2. Canny 에지 검출

1번 영상에 대해 $\sigma = 1, 3, 5, 7$ 로 변화시킨 Canny 에지 검출을 수행하고 소스 코드와 결과 영상을 제시.
Threshold를 적절히 조절. σ 를 조절하는 경우 Canny 함수 호출하기 전에 Gaussian convolution을 먼저 해야함.

```
import cv2 as cv
import matplotlib.pyplot as plt

img = cv.imread('bed.png')
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

sigma_values = [1, 3, 5, 7]
canny_results = []

for sigma in sigma_values:
    # Gaussian Blur: sigma만 설정, 커널은 자동 선택
    blurred = cv.GaussianBlur(gray, (0, 0), sigmaX=sigma, sigmaY=sigma)

    if sigma == 7:
        threshold_low, threshold_high = 15, 40
    else:
        threshold_low = max(10, 30 - sigma * 3)
        threshold_high = max(30, 60 - sigma * 5)

    edges = cv.Canny(blurred, threshold_low, threshold_high)
    canny_results.append((sigma, threshold_low, threshold_high, edges))

# 시각화
plt.figure(figsize=(9, 6))

plt.subplot(2, 3, 1)
plt.title('Original')
plt.imshow(img)
plt.axis('off')

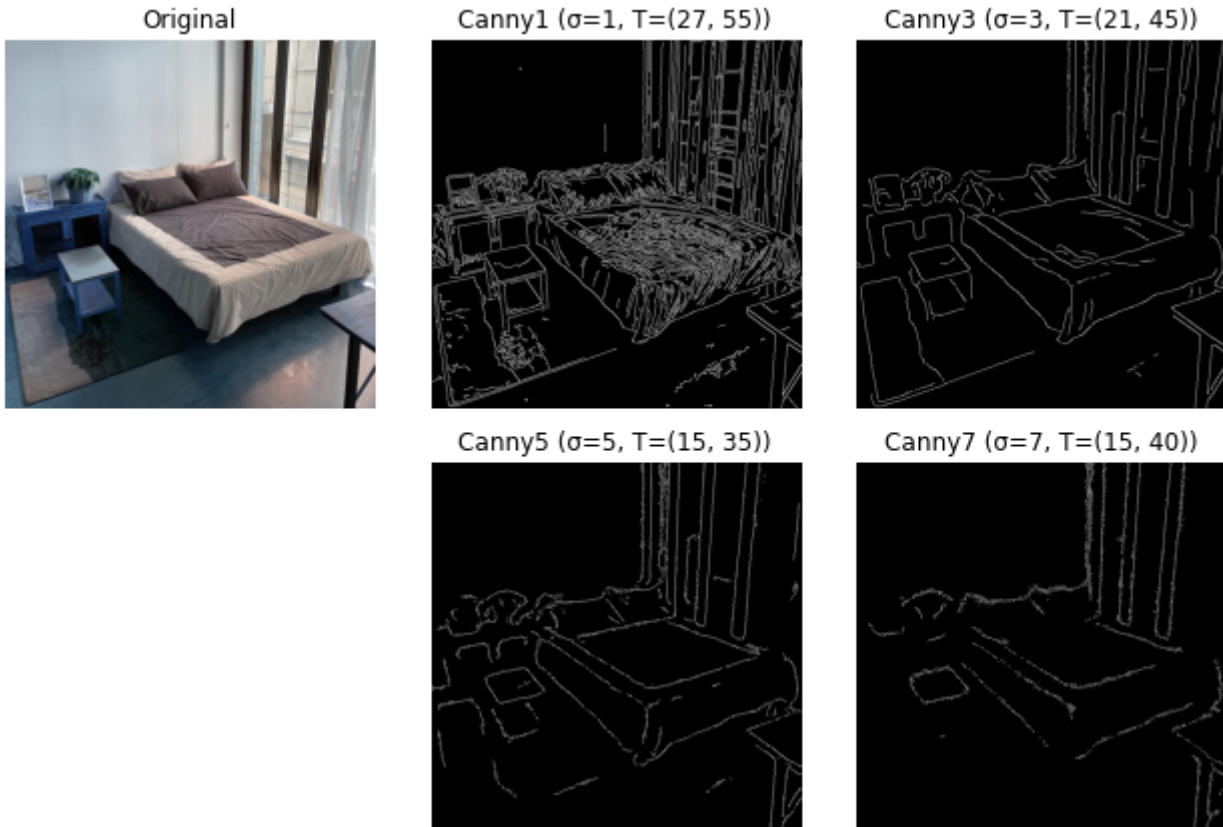
plt.subplot(2, 3, 2)
plt.title(f'Canny1 ( $\sigma=1$ , T=({canny_results[0][1]}, {canny_results[0][2]}))')
plt.imshow(canny_results[0][3], cmap='gray')
plt.axis('off')

plt.subplot(2, 3, 3)
plt.title(f'Canny3 ( $\sigma=3$ , T=({canny_results[1][1]}, {canny_results[1][2]}))')
plt.imshow(canny_results[1][3], cmap='gray')
plt.axis('off')

plt.subplot(2, 3, 5)
plt.title(f'Canny5 ( $\sigma=5$ , T=({canny_results[2][1]}, {canny_results[2][2]}))')
plt.imshow(canny_results[2][3], cmap='gray')
plt.axis('off')
```

```
plt.subplot(2, 3, 6)
plt.title(f'Canny7 ( $\sigma=7$ ,  $T=({canny\_results[3][1]}, {canny\_results[3][2]}))$ ')
plt.imshow(canny_results[3][3], cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```



이 코드는 입력 영상 'bed.png'에 대해 Canny 에지 검출을 수행하고, Gaussian 필터의 표준편차 값 σ 를 변화시키면서 에지 검출 결과가 어떻게 달라지는지를 시각적으로 비교할 수 있도록 구성되어 있습니다. 먼저 영상을 그레이스케일로 변환한 뒤, σ 값을 1, 3, 5, 7로 설정하여 Gaussian Blur를 적용했습니다. 이때 커널 크기는 자동으로 설정되며, σ 값이 커질수록 더 강한 흐림 효과가 적용됩니다. 흐림 처리된 영상에 대해 Canny 에지 검출을 수행하는데, σ 가 커질수록 에지가 흐려지기 때문에 약한 에지가 사라지지 않도록 threshold 값을 점차 낮추는 방식으로 조정했습니다. 특히 $\sigma=7$ 인 경우에는 너무 많은 에지가 나타나는 것을 방지하기 위해 threshold 값을 별도로 설정했습니다. 그 결과를 원본 영상과 함께 4개의 σ 값에 대응하는 Canny 결과 영상으로 시각화하여, σ 에 따른 에지 검출 민감도의 변화를 직관적으로 확인할 수 있도록 했습니다.

3. Canny edge에서 방향 검출

- 256x256 크기 영상에서 중심이 (128, 128) 이고 반경이 80 픽셀인 원을 생성. 배경의 색상은 50, 원의 색상은 200으로 지정.
- $s = 1$ 인 Canny 방식으로 에지를 검출
- 각 에지점의 방향을 검출하는 기능을 추가하여 각도가 $30^{\circ} \sim 60^{\circ}$ 인 픽셀들만 검출하여 디스플레이 함

```
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt

# 256x256 영상 생성 (배경: 50, 원: 200)
img = np.full((256, 256), 50, dtype=np.uint8)
cv.circle(img, center=(128, 128), radius=80, color=200, thickness=-1)

# Canny edge 검출
edges = cv.Canny(img, threshold1=50, threshold2=150)

# 방향 계산을 위한 Sobel
gx = cv.Sobel(img, cv.CV_64F, 1, 0, ksize=3)
gy = cv.Sobel(img, cv.CV_64F, 0, 1, ksize=3)

# 에지 방향 계산 (라디안 기준)
theta = np.arctan2(gy, gx) # 각도 범위:  $[-\pi, \pi]$ 
theta_deg = np.degrees(theta) # 라디안 → 도 (degree)

# 각도를 [0, 360] 범위로 정규화
theta_deg = (theta_deg + 360) % 360

# 마스크 생성: Canny edge 이면서, 각도가 30~60도인 부분만
mask = (edges > 0) & (theta_deg >= 30) & (theta_deg <= 60)

# 디스플레이용 이미지 만들기
output = np.zeros_like(img)
output[mask] = 255

# 시각화
plt.figure(figsize=(9, 3))

plt.subplot(1, 3, 1)
plt.title('Original Circle')
plt.imshow(img, cmap='gray')
plt.axis('off')

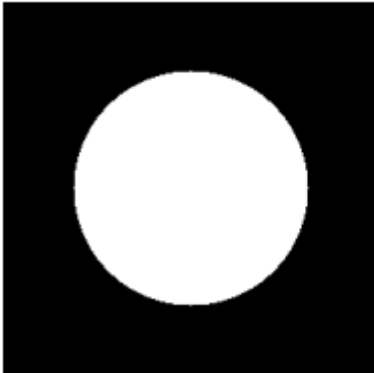
plt.subplot(1, 3, 2)
plt.title('Canny Edge (s=1)')
plt.imshow(edges, cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title('Edges (Angle 30°~60°)')
plt.imshow(output, cmap='gray')
plt.axis('off')
```

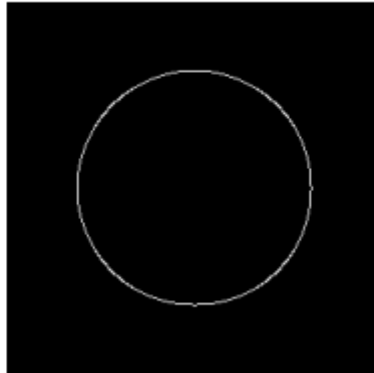
```
plt.imshow(output, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Original Circle



Canny Edge (s=1)



Edges (Angle 30°~60°)



이 코드는 256×256 크기의 인공 영상에 대해 Canny 에지 검출을 수행하고, 검출된 에지 중에서 방향이 특정 각도 범위(30°~60°)에 해당하는 부분만을 추출하여 시각화하는 과정을 포함하고 있습니다. 먼저 배경 픽셀 값이 50이고 중심이 (128, 128), 반지름이 80이며 픽셀 값이 200인 원을 생성하여 테스트용 영상을 만듭니다. 이후 해당 영상에 대해 Canny 에지 검출을 수행하여 에지 위치를 확인하고, 동시에 Sobel 연산을 통해 수평(Gx) 및 수직(Gy) 방향의 미분 값을 구합니다. 이 값을 이용해 각 픽셀의 에지 방향을 $\arctan2$ 함수로 계산하고, 라디안 값을 도(degree)로 변환한 뒤 0도에서 360도 사이로 정규화합니다. 그런 다음 Canny로 검출된 에지 중에서 방향이 30도 이상 60도 이하인 픽셀만을 선택하여 마스크를 생성하고, 이를 통해 해당 방향의 에지만 포함하는 출력 이미지를 구성합니다. 최종적으로 원본 영상, 전체 Canny 에지 영상, 방향이 30°~60°인 에지 결과를 나란히 출력하여 시각적으로 비교할 수 있도록 했습니다.

4. 실습 1~4의 코드와 결과를 제출함

실습 1

```
import cv2 as cv
import matplotlib.pyplot as plt

gray = cv.imread('pattern.png', cv.IMREAD_GRAYSCALE)

# 반경이 8인 disk(15x15) structuring element
kernel = cv.getStructuringElement(cv.MORPH_ELLIPSE, (15, 15))

# 형태학적 연산
erosion = cv.erode(gray, kernel, iterations=1)
dilation = cv.dilate(gray, kernel, iterations=1)
opening = cv.morphologyEx(gray, cv.MORPH_OPEN, kernel)
closing = cv.morphologyEx(gray, cv.MORPH_CLOSE, kernel)

# 시각화
plt.figure(figsize=(12, 8))

plt.subplot(2,3,1)
plt.title('Original (A)')
plt.imshow(gray, cmap='gray')
plt.axis('off')

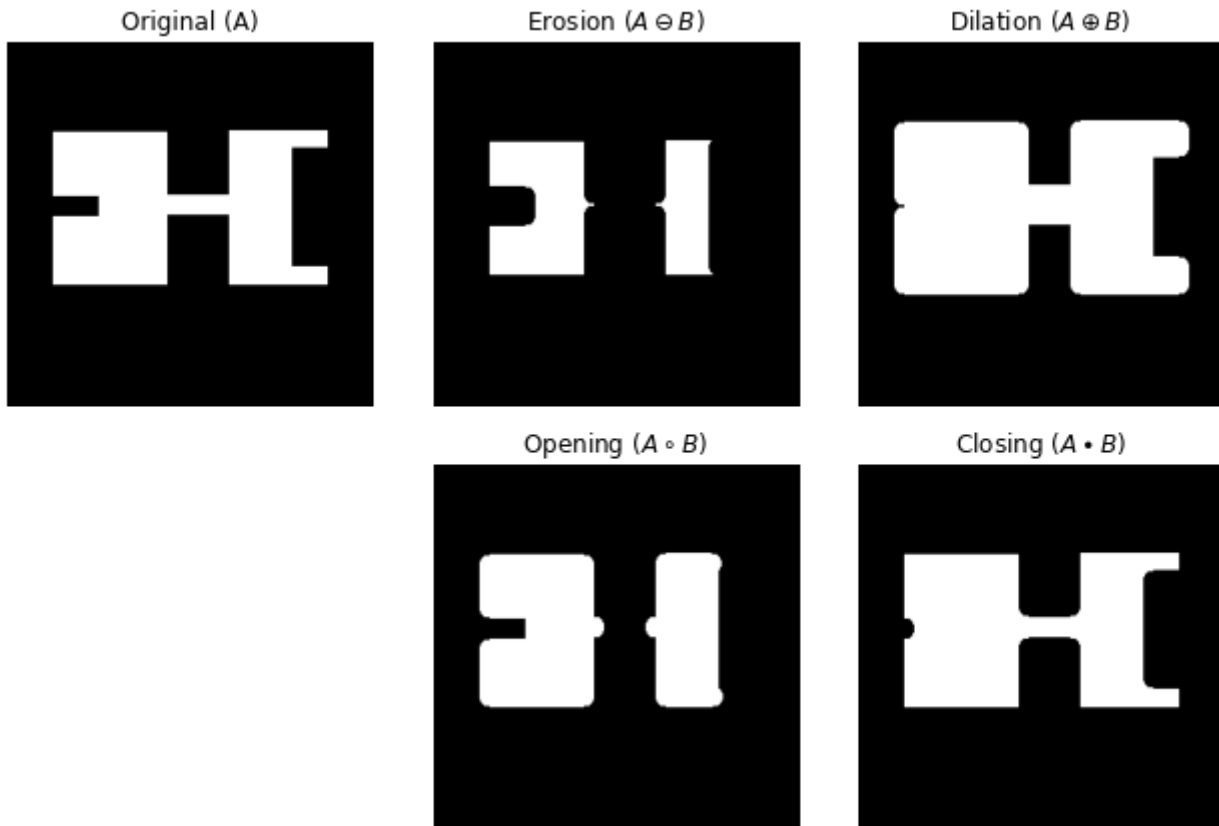
plt.subplot(2,3,2)
plt.title(r'Erosion ($A \ominus B$)')
plt.imshow(erosion, cmap='gray')
plt.axis('off')

plt.subplot(2,3,3)
plt.title(r'Dilation ($A \oplus B$)')
plt.imshow(dilation, cmap='gray')
plt.axis('off')

plt.subplot(2,3,5)
plt.title(r'Opening ($A \circ B$)')
plt.imshow(opening, cmap='gray')
plt.axis('off')

plt.subplot(2,3,6)
plt.title(r'Closing ($A \bullet B$)')
plt.imshow(closing, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```



실습 2

```
import cv2 as cv
import matplotlib.pyplot as plt

img = cv.imread('0907.png', cv.IMREAD_GRAYSCALE)

# 3x3 cross-shaped structuring element
kernel = cv.getStructuringElement(cv.MORPH_CROSS, (3, 3))

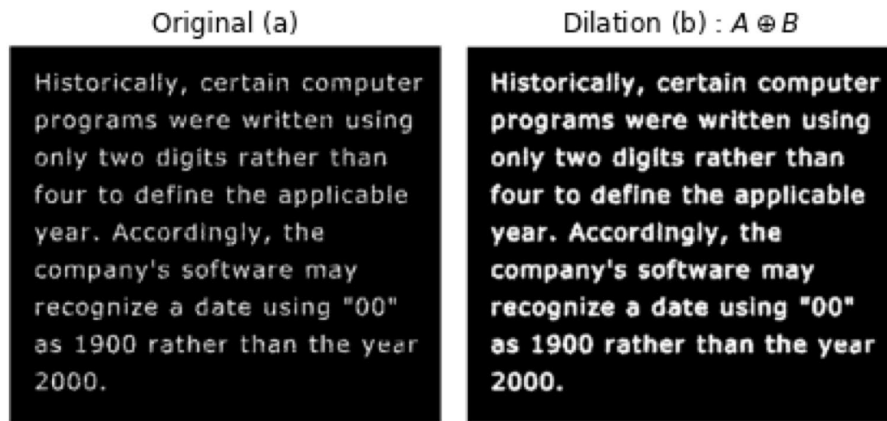
# Dilation 수행
dilated = cv.dilate(img, kernel, iterations=1)

# 시각화
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.title('Original (a)')
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title(r'Dilation (b) : $A \oplus B$')
plt.imshow(dilated, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```



실습 3

```
import cv2 as cv
import matplotlib.pyplot as plt

img = cv.imread('0911.png', cv.IMREAD_GRAYSCALE)

# 3x3 정사각형 structuring element
kernel = cv.getStructuringElement(cv.MORPH_RECT, (3, 3))

# Opening (Erosion → Dilation)
opening = cv.morphologyEx(img, cv.MORPH_OPEN, kernel)

# Opening 후 Closing (Dilation → Erosion)
closing_after_opening = cv.morphologyEx(opening, cv.MORPH_CLOSE, kernel)

# 시각화
plt.figure(figsize=(12, 6))

plt.subplot(1, 3, 1)
plt.title('Original (a)')
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.title(r'Opening (c):  $A \circ B$ ')
plt.imshow(opening, cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title(r'Opening→Closing (e):  $(A \circ B) \bullet B$ ')
plt.imshow(closing_after_opening, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```



실습 4

```
import cv2 as cv
import matplotlib.pyplot as plt

img = cv.imread('0914.png', cv.IMREAD_GRAYSCALE)

# 3x3 정사각형 structuring element
kernel = cv.getStructuringElement(cv.MORPH_RECT, (3, 3))

# Erosion 수행
eroded = cv.erode(img, kernel, iterations=1)

# 원본 - Erosion으로 경계 추출
boundary = cv.subtract(img, eroded)

# 시각화
plt.figure(figsize=(8, 4))

plt.subplot(1, 2, 1)
plt.title('Original')
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title(r'Boundary: $A - (A \ominus B)$')
plt.imshow(boundary, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```

